Program 1

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>

int main()
{
    FILE *f1;
    int c;
    char str[50];
    int i = 0, k, num, linecount = 0, iskw;
    const char *keywords[] = {"for", "while", "do", "int", "float", "char", "double",
                              "static", "switch", "case", "if", "else", "return",
                              "void", "long", "short", "break", "continue"};
    int nkw = sizeof(keywords) / sizeof(keywords[0]);

    f1 = fopen("input.txt", "r");
    if (f1 == NULL)
    {
        printf("Cannot open input.txt\n");
        return 1;
    }

    while ((c = getc(f1)) != EOF)
    {
        if (isdigit(c))
        {
            num = c - '0';
            c = getc(f1);
            while (isdigit(c))
            {
                num = num * 10 + (c - '0');
                c = getc(f1);
            }
            printf("%d is a number\n", num);
            ungetc(c, f1);
        }
        else if (isalpha(c))
        {
            i = 0;
            str[i++] = c;
            c = getc(f1);
            while ((isdigit(c) || isalpha(c) || c == '_' || c == '$') && i < 48)
            {
                str[i++] = c;
                c = getc(f1);
            }
            str[i] = '\0';

            iskw = 0;
            for (k = 0; k < nkw; k++)
                if (strcmp(keywords[k], str) == 0)
                {
                    iskw = 1;
                    break;
                }

            if (iskw)
                printf("%s is a keyword \n", str);
            else
                printf("%s is an identifier\n", str);
            ungetc(c, f1);
        }
        else if (c == ' ' || c == '\t')
            printf("\n");
        else if (c == '\n')
            linecount++;
        else
            printf("%c is a special symbol\n", c);
    }
    printf("Total no of lines are :%d\n", linecount);
    fclose(f1);
    return 0;
}
```

Program 6

Lex

```
%{
#include "y.tab.h"
%}
%%
[0-9]+                  { return NUMBER; }
[a-zA-Z][a-zA-Z0-9]*    { return ID; }
[ \t]                   ;
\n                      { return 0; }
.                       { return yytext[0]; }
%%
int yywrap()
{
    return 1;
}
```

Yacc

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex(void);
int yyerror(const char *s);
%}
%token NUMBER ID
%left '+' '-'
%left '*' '/'
%%
exp : exp '+' exp
    | exp '-' exp
    | exp '*' exp
    | exp '/' exp
    | '(' exp ')'
    | NUMBER
    | ID
    ;
%%
int main()
{
    printf("Enter the expression:");
    yyparse();
    printf("Valid expression\n");
    return 0;
}
int yyerror(const char *s)
{
    printf("invalid expression\n");
    exit(1);
    return 0;
}
```

Program 8

Lex

```
%{
#include "y.tab.h"
#include <stdio.h>
#include <stdlib.h>
%}
%%
[0-9]+      { yylval.ival = atoi(yytext); return NUMBER; }
[ \t]       ;
\n          { return '\n'; }
.           { return yytext[0]; }
%%
```

Yacc

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex(void);
int yywrap(void);
void yyerror(const char *s);
%}
%union { int ival; }
%token <ival> NUMBER
%type <ival> exp
%left '+' '-'
%left '*' '/'
%%
st  : exp '\n'          { printf("Answer=%d\n", $1); }
    ;
exp : exp '+' exp       { $$ = $1 + $3; }
    | exp '-' exp       { $$ = $1 - $3; }
    | exp '*' exp       { $$ = $1 * $3; }
    | exp '/' exp       {
                            if ($3 == 0)
                            {
                                printf("Division by zero\n");
                                exit(1);
                            }
                            $$ = $1 / $3;
                        }
    | '(' exp ')'       { $$ = $2; }
    | NUMBER            { $$ = $1; }
    ;
%%
int main()
{
    printf("Enter the expression:");
    yyparse();
    return 0;
}
int yywrap()
{
    return 1;
}
void yyerror(const char *s)
{
    printf("Error\n");
}
```

Program 9

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
#include<stdlib.h>

int n, m = 0;
char a[20][20], f[50];
int visited_first[256], visited_follow[256];

void add(char c);
void first(char c);
void follow(char c);

int main()
{
    int i, z;
    char c;

    printf("Enter the No.of productions:\n");
    scanf("%d", &n);
    if (n < 1 || n > 20)
    {
        printf("Number of productions must be between 1 and 20\n");
```

```
        return 1;
    }
    printf("Enter the productions:\n");
    for (i = 0; i < n; i++)
        scanf("%19s", a[i]);
    do
    {
        printf("Enter the elements whose first and follow is to be found:");
        scanf(" %c", &c);

        m = 0;
        memset(visited_first, 0, sizeof(visited_first));
        first(c);
        printf("First(%c)={", c);
        for (i = 0; i < m; i++)
            printf("%c", f[i]);
        printf("}\n");

        m = 0;
        memset(visited_first, 0, sizeof(visited_first));
        memset(visited_follow, 0, sizeof(visited_follow));
        follow(c);
        printf("Follow(%c)={", c);
        for (i = 0; i < m; i++)
            printf("%c", f[i]);
        printf("}\n");

        printf("Continue (0/1)?");
        scanf("%d", &z);
    } while (z == 1);
    return 0;
}

void add(char c)
{
    int k;
    for (k = 0; k < m; k++)
        if (f[k] == c)
            return;
    f[m++] = c;
}

void first(char c)
{
    int k;
    if (!isupper((unsigned char)c))
    {
        add(c);
        return;
    }
    if (visited_first[(unsigned char)c])
        return;
    visited_first[(unsigned char)c] = 1;
    for (k = 0; k < n; k++)
    {
        if (a[k][0] == c && a[k][2] != '$' && a[k][2] != '\0')
            first(a[k][2]);
    }
}

void follow(char c)
{
    int i, j;
    if (visited_follow[(unsigned char)c])
        return;
    visited_follow[(unsigned char)c] = 1;
    if (a[0][0] == c)
        add('$');
    for (i = 0; i < n; i++)
    {
        for (j = 2; j < (int)strlen(a[i]); j++)
        {
            if (a[i][j] == c)
            {
                if (a[i][j + 1] != '\0')
                    first(a[i][j + 1]);
```

```
                else if (c != a[i][0])
                    follow(a[i][0]);
            }
        }
    }
}
```

Program 10

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>

char in[100];
int i = 0, err = 0;

void E(void), Ep(void), T(void), Tp(void), F(void);

int main()
{
    char buf[100];
    int k, len = 0;

    printf("Enter expression:");
    if (fgets(buf, sizeof(buf), stdin) == NULL)
        return 1;
    buf[strcspn(buf, "\n")] = '\0';

    /* remove blanks from the input */
    for (k = 0; buf[k] != '\0'; k++)
        if (buf[k] != ' ' && buf[k] != '\t')
            in[len++] = buf[k];
    in[len] = '\0';

    E();
    if (!err && in[i] == '\0')
        printf("Accepted\n");
    else
        printf("Rejected\n");
    return 0;
}

void E(void)
{
    T();
    Ep();
}

void Ep(void)
{
    if (in[i] == '+')
    {
        i++;
        T();
        Ep();
    }
}

void T(void)
{
    F();
    Tp();
}

void Tp(void)
{
    if (in[i] == '*')
    {
        i++;
        F();
        Tp();
    }
}
```

```
void F(void)
{
    if (in[i] == '(')
    {
        i++;
        E();
        if (in[i] == ')')
            i++;
        else
            err = 1;
    }
    else if (isalpha((unsigned char)in[i]))
    {
        i++;
        while (isalnum((unsigned char)in[i]) || in[i] == '_')
            i++;
    }
    else
        err = 1;
}
```

Program 11

```
#include<stdio.h>
#include<string.h>

char op[4], arg1[16], arg2[16], result[16];

int main()
{
    FILE *fp1, *fp2;

    fp1 = fopen("input.txt", "r");
    if (fp1 == NULL)
    {
        printf("Cannot open input.txt\n");
        return 1;
    }
    fp2 = fopen("output.txt", "w");
    if (fp2 == NULL)
    {
        printf("Cannot create output.txt\n");
        fclose(fp1);
        return 1;
    }

    while (fscanf(fp1, "%3s%15s%15s%15s", op, arg1, arg2, result) == 4)
    {
        if (strcmp(op, "+") == 0)
        {
            fprintf(fp2, "MOV R0,%s\n", arg1);
            fprintf(fp2, "ADD R0,%s\n", arg2);
            fprintf(fp2, "MOV %s,R0\n", result);
        }
        else if (strcmp(op, "*") == 0)
        {
            fprintf(fp2, "MOV R0,%s\n", arg1);
            fprintf(fp2, "MUL R0,%s\n", arg2);
            fprintf(fp2, "MOV %s,R0\n", result);
        }
        else if (strcmp(op, "-") == 0)
        {
            fprintf(fp2, "MOV R0,%s\n", arg1);
            fprintf(fp2, "SUB R0,%s\n", arg2);
            fprintf(fp2, "MOV %s,R0\n", result);
        }
        else if (strcmp(op, "/") == 0)
        {
            fprintf(fp2, "MOV R0,%s\n", arg1);
            fprintf(fp2, "DIV R0,%s\n", arg2);
            fprintf(fp2, "MOV %s,R0\n", result);
        }
        else if (strcmp(op, "=") == 0)
```

```
        {
            fprintf(fp2, "MOV R0,%s\n", arg1);
            fprintf(fp2, "MOV %s,R0\n", result);
        }
    }
    fclose(fp1);
    fclose(fp2);
    return 0;
}
```

Program 12

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>

char in[50], st[50];
int ip = 0, sp = 0, n;

void reduce()
{
    int f = 1;
    while (f)
    {
        f = 0;
        char t;
        if (st[sp - 1] == 'a' || st[sp - 1] == 'b' || st[sp - 1] == 'c')
        {
            t = st[sp - 1];
            st[sp - 1] = 'E';
            printf("\n$%s\t\t%s\t\tE->%c", st, in + ip, t);
            f = 1;
        }
        if (sp >= 3 && st[sp - 3] == 'E' && strchr("+-*/", st[sp - 2]) && st[sp - 1] == 'E')
        {
            char op = st[sp - 2];
            st[sp - 3] = 'E';
            st[sp - 2] = st[sp - 1] = '\0';
            sp -= 2;
            printf("\n$%s\t\t%s\t\tE->E%cE", st, in + ip, op);
            f = 1;
        }
    }
}

int main()
{
    printf("\nSHIFT REDUCE PARSER\n\nGrammar:\nE->E+E\nE->E*E\nE->E/E\nE->a/b/c\n");
    printf("\nEnter input: ");
    if (scanf("%47s", in) != 1)
        return 1;
    n = strlen(in);
    in[n++] = '$';
    in[n] = '\0';
    printf("\nStack\t\tInput\t\tAction\n$\t\t%s\t\t--", in);
    while (1)
    {
        if (!(isalpha((unsigned char)in[ip]) || strchr("*/+$", in[ip])))
        {
            printf("\n$%s\t\t%s\t\tREJECT", st, in + ip);
            break;
        }
        st[sp++] = in[ip];
        st[sp] = '\0';
        printf("\n$%s\t\t%s\t\tshift %c", st, in + ++ip, st[sp - 1]);
        reduce();
        if (!strcmp(st, "E") && in[ip] == '$')
        {
            printf("\n$%s\t\t%s\t\tACCEPT", st, in + ip);
            break;
        }
        if (ip >= n && strcmp(st, "E"))
```

```
        {
            printf("\n$%s\t\t%s\t\tREJECT", st, in + ip);
            break;
        }
    }
    printf("\n");
    return 0;
}
```

Program 13

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
#include<stdlib.h>

struct expr
{
    char op[4], op1[16], op2[16], res[16];
    int flag;
} arr[10];
int n;

void change(int p, char *res)
{
    for (int i = p + 1; i < n; i++)
    {
        if (!strcmp(arr[p].res, arr[i].op1))
            strcpy(arr[i].op1, res);
        if (!strcmp(arr[p].res, arr[i].op2))
            strcpy(arr[i].op2, res);
        if (!strcmp(arr[p].res, arr[i].res))
            break;
    }
}

void constant()
{
    for (int i = 0; i < n; i++)
    {
        int is_assign = !strcmp(arr[i].op, "=");
        if ((isdigit((unsigned char)arr[i].op1[0]) &&
             (is_assign || isdigit((unsigned char)arr[i].op2[0]))))
        {
            int op1 = atoi(arr[i].op1), op2 = atoi(arr[i].op2), res = 0, ok = 1;
            switch (arr[i].op[0])
            {
                case '+': res = op1 + op2; break;
                case '-': res = op1 - op2; break;
                case '*': res = op1 * op2; break;
                case '/':
                    if (op2 == 0)
                        ok = 0;
                    else
                        res = op1 / op2;
                    break;
                case '=': res = op1; break;
                default: ok = 0;
            }
            if (ok)
            {
                char res1[16];
                sprintf(res1, "%d", res);
                arr[i].flag = 1;
                change(i, res1);
            }
        }
    }
}

int main()
{
    printf("Enter number of expressions:");
```

```
    if (scanf("%d", &n) != 1 || n < 1 || n > 10)
    {
        printf("Number of expressions must be between 1 and 10\n");
        return 1;
    }
    printf("Enter expressions as input:\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%3s%15s%15s%15s", arr[i].op, arr[i].op1, arr[i].op2, arr[i].res);
        arr[i].flag = 0;
    }
    constant();
    printf("\nOptimized code:\n");
    for (int i = 0; i < n; i++)
        if (!arr[i].flag)
            printf("%s %s %s %s\n", arr[i].op, arr[i].op1, arr[i].op2, arr[i].res);
    return 0;
}
```